

Unit 2

SMD Practical + Oral Exam Notes

UNIT 2 — OBJECT ORIENTED ANALYSIS

Topic 1: Use Case Modeling — Actors, Classification & Generalization

1. Simple Exam Definition:

Use Case Modeling identifies **who** uses the system (Actors) and **what** they can do with it (Use Cases). It captures functional requirements from the user's perspective.

2. Core Concept Explanation:

Actor:

- Anything that interacts with the system from **outside** the system boundary.
- Can be a person (Student, Admin) or an external system (Payment Gateway, Email Server).

Actor Classification (3 types — examiners love this!):

1. **Primary Actor:** Directly uses the system to achieve a goal. (e.g., Student enrolls in course.)
2. **Secondary Actor:** Supports the system but doesn't initiate use cases. (e.g., Email Server sends confirmation.)
3. **Off-Stage Actor:** Has a business interest but rarely interacts directly. (e.g., University Management views reports.)

Actor Generalization:

When two actors share common behaviour, a general actor is created. Specialized actors inherit the general actor's use cases.

- Example: `User` is a general actor. `Student` and `Professor` are specialized actors inheriting `Login` from `User`.

Use Case:

- A complete sequence of actions that produces a **result of value** to the actor.
- Named using **Verb + Noun** format: `EnrollInCourse`, `GenerateReport`, `ViewGrades`.

System Boundary:

- A rectangle showing the scope of the system. Actors are outside; Use Cases are inside.

3. Real Practical Example (Student Management System):

- **Primary Actor:** Student (Enroll in Course, View Grades).
- **Secondary Actor:** Email Service (sends confirmation after enrollment).
- **Off-Stage Actor:** University Administration (reads reports).
- **Actor Generalization:** `User` → `Student`, `Professor`.

4. Examiner Viva Questions:

- What is an Actor? Can an actor be a non-human?
- What are the 3 types of actors?
- What is Actor Generalization?
- Why must use cases be named using verbs?

5. Strong Viva Answers:

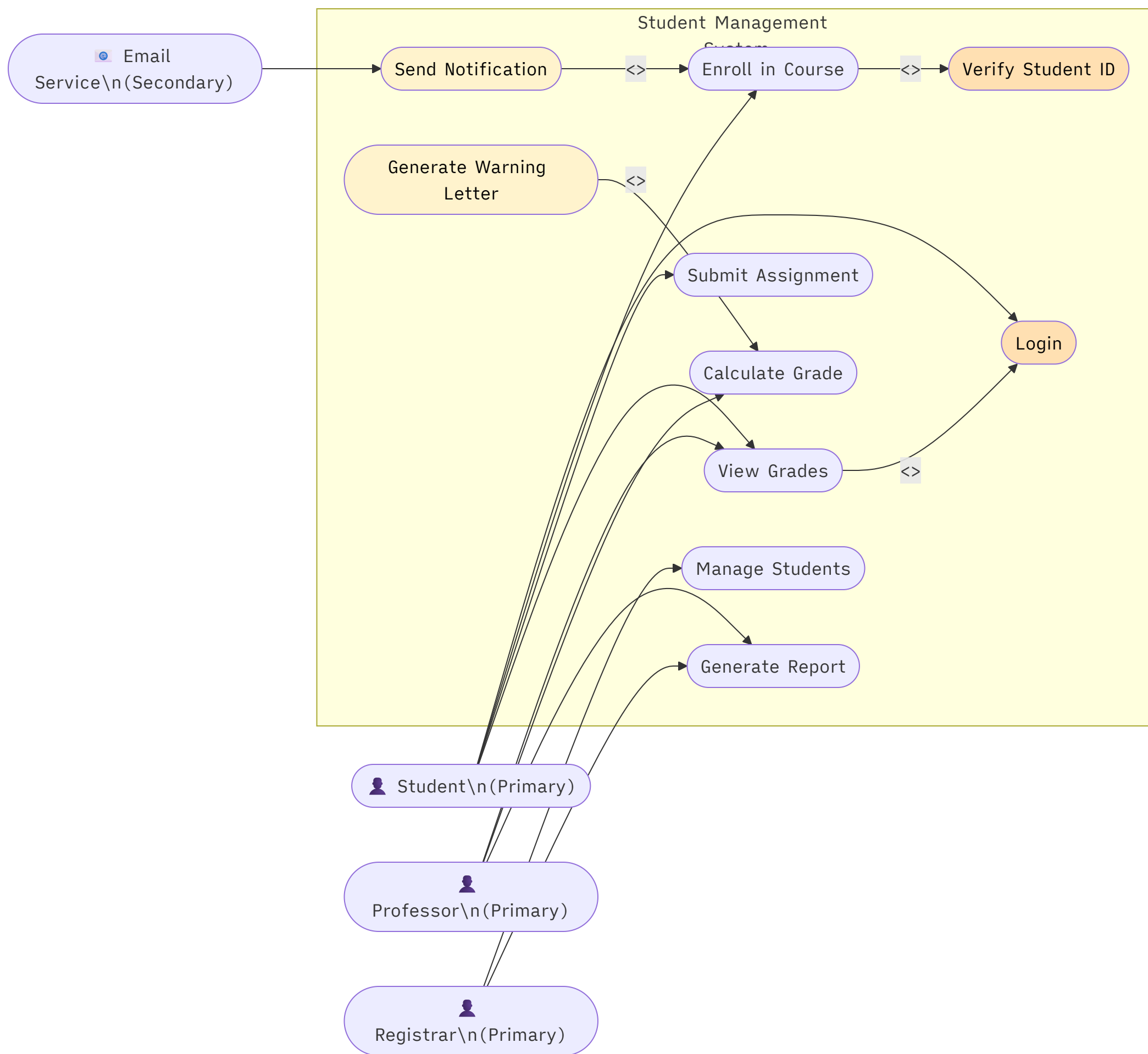
- *Answer:* An actor is any entity outside the system that interacts with it. Yes, a non-human actor like a Payment Gateway or Email Server is valid.
- *Answer:* Primary actors initiate use cases. Secondary actors support the system. Off-stage actors have an indirect business interest.
- *Answer:* Use cases are named using Verb+Noun because they represent **actions**, not things. `EnrollInCourse` is better than `Enrollment`.

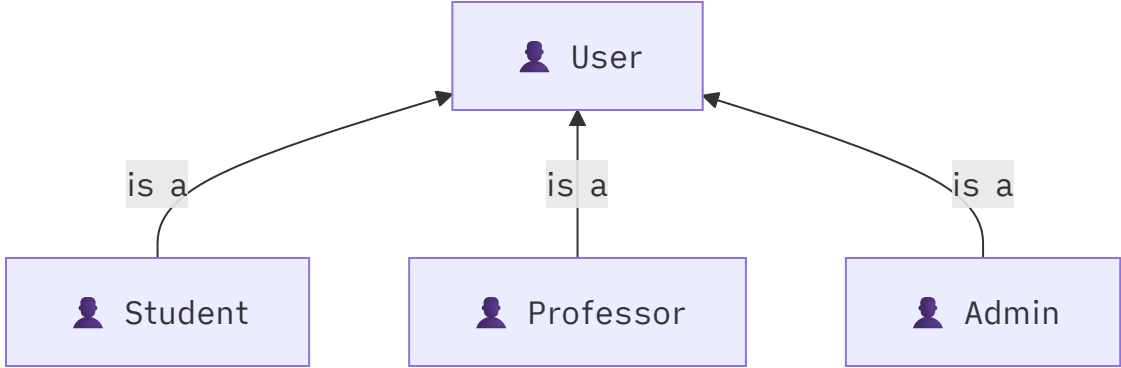
6. Common Mistakes Students Make:

- Placing actors inside the system boundary box.
- Naming use cases as nouns (e.g., "Enrollment" instead of "EnrollInCourse").
- Forgetting that secondary actors and off-stage actors exist.

9. Difference Tables:

Type	Initiates Use Case?	Example
Primary	Yes	Student enrolls
Secondary	No (supports)	Email Server notifies
Off-Stage	Rarely	Management reads reports





💡 **Exam tip:** `User` provides `Login` use case. Both `Student` and `Professor` inherit it through generalization. Draw the generalization arrow from child actor to parent actor (hollow triangle at parent).

Topic 2: Writing Formal Use Case Specification

1. Simple Exam Definition:

A Formal Use Case Specification is a structured text document that precisely describes a use case – its actors, preconditions, main flow, alternative flows, and postconditions.

2. Core Concept Explanation:

Every use case must be documented formally. Examiners often ask you to write this for the practical file.

Standard Template:

Field	Description
Use Case Name	EnrollInCourse
Actor(s)	Student (Primary), System (Secondary)
Precondition	Student is logged in; Course has available seats
Main Flow (Basic Path)	1. Student selects course → 2. System verifies ID → 3. System deducts fee → 4. System confirms enrollment
Alternate Flow	3a. Fee payment fails → System shows error → Student retries
Exception Flow	2a. Invalid Student ID → System rejects and logs out
Postcondition	Student is enrolled; Seat count decremented; Confirmation email sent

3. Real Practical Example (Student Management System):

Use Case: Login

- **Actor:** Student
- **Precondition:** Student has valid credentials
- **Main Flow:** 1. Student enters username and password → 2. System validates → 3. System grants access
- **Alternate Flow:** 2a. Invalid password → System shows error → Student retries (max 3 attempts)
- **Exception:** 2b. Account locked → System redirects to admin
- **Postcondition:** Student is authenticated and redirected to dashboard

4. Examiner Viva Questions:

- What is a precondition in a use case specification?
- What is the difference between Alternate Flow and Exception Flow?

5. Strong Viva Answers:

- *Answer:* Precondition is the state the system must be in before the use case begins. Postcondition is the state after successful completion.
- *Answer:* Alternate Flow is an accepted deviation from the main path (e.g., pay by card instead of UPI). Exception Flow is an error or failure path (e.g., system is down).

13. Practical File Expectations:

Your journal MUST contain the formal use case specification for at least 2 use cases (e.g., Login and Enroll in Course). Examiners will ask you to explain your main flow.

Topic 3: Include vs Extend (CRITICAL FOR VIVA)

1. Simple Exam Definition:

- `<<include>>`: A base use case **always** calls another use case. It is **mandatory**.
- `<<extend>>`: A use case **optionally** extends the behaviour of a base use case under a condition.

2. Core Concept Explanation:

Include:

- Use Case A includes Use Case B. B always happens when A is executed.
- Think of it like a **function call** — A always calls B.
- Arrow direction: **From A → To B** (Base points to the included helper).

Extend:

- Use Case B extends Use Case A. B only happens if a condition is met.
- Think of it like an **optional add-on** — B attaches itself to A conditionally.
- Arrow direction: **From B → To A** (The extension points to the base it extends).

3. Real Practical Example (Student Management System):

- **Include:** `EnrollInCourse` `<<include>>` `VerifyStudentID` → Cannot enroll without ID verification (ALWAYS happens).
- **Extend:** `GenerateWarningLetter` `<<extend>>` `CalculateGrade` → Only if grade < 40 (OPTIONAL).
- **Include:** `ViewGrades` `<<include>>` `Login` → Cannot view grades without logging in.

4. Examiner Viva Questions:

- Arrow direction for `<<include>>`?
- Arrow direction for `<<extend>>`?
- Is Login an include or extend?

5. Strong Viva Answers:

- Answer: For <<include>>, the arrow goes FROM the base use case TO the included use case.
- Answer: For <<extend>>, the arrow goes FROM the extending use case TO the base use case.
- Answer: Login is typically an <<include>> because viewing any secure page ALWAYS requires login first.

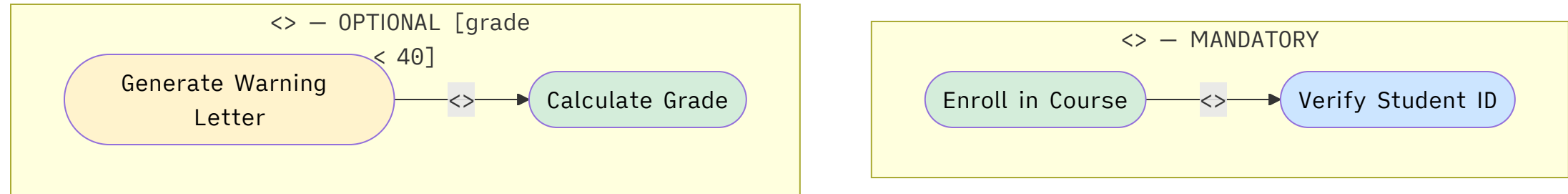
6. Common Mistakes — #1 Most Common Mistake:

Drawing the extend arrow in the WRONG direction. The arrow points TO the base, not to the extension.

9. Difference Tables:

Feature	<<include>>	<<extend>>
Nature	Mandatory	Optional / Conditional
Arrow Direction	Base → Included	Extension → Base
When executes?	Always	Only if condition met
Who initiates?	Base use case	Extension use case
Example	EnrollInCourse → VerifyID	GenerateWarning → CalculateGrade

Visual: Include vs Extend — Side by Side



⚠ CRITICAL RULE: In <<extend>>, the optional use case arrow points TO the base. Think: "The warning letter *looks at* the grade calculation to know when to attach itself."

10. Memory Tricks:

- Include = Inside (always part of the flow) → **Arrow OUT from base.**
- Extend = **E**xtra (optional, attaches from outside) → **Arrow IN to base.**
- Rule: Both arrows are **dashed** lines with <<stereotype>> label.

Topic 4: Class Modeling & Relationships (CRITICAL FOR VIVA)

1. Simple Exam Definition:

Class Modeling identifies the static structure of the system — the classes (blueprints), their attributes (data), methods (behaviour), and the relationships between them.

2. Core Concept Explanation — The 4 Relationships:

1. Association

- A general, structural link between two classes.
- Can be unidirectional (one arrow) or bidirectional (no arrow or both arrows).
- Has **multiplicity** (e.g., `1`, `*`, `1..*`, `0..1`).
- Example: `Student` enrolls in `Course` (many-to-many: `*` to `*`).

2. Aggregation (Weak "Part-of")

- A whole-part relationship where the **part can exist without the whole**.
- Notation: **Hollow diamond** at the whole's end.
- Example: `Course` contains `Syllabus`. If the course is deleted, the syllabus document still exists independently.

3. Composition (Strong "Part-of")

- A whole-part relationship where the **part CANNOT exist without the whole**.
- Notation: **Filled diamond** at the whole's end.
- Example: `Student` has `AcademicRecord`. If the student is deleted, the record is meaningless and deleted too.

4. Generalization (Inheritance / "Is-a")

- A parent-child (superclass-subclass) relationship.
- Notation: **Solid line with a hollow triangle** pointing to the parent.
- Example: `User` ← `Student` (Student is a User and inherits login(), etc.)

3. Real Practical Example (Student Management System):

Relationship	Example	Why?
Association	Student ↔ Course	A student enrolls in courses; a course has students
Aggregation	Course ◇— Syllabus	Syllabus exists independently
Composition	Student ◆— AcademicRecord	Record has no meaning without the student
Generalization	User ◁▷ Student, Professor	Both are types of User

4. Multiplicity — What Examiners Ask:

Notation	Meaning
<code>1</code>	Exactly one
<code>*</code>	Zero or more
<code>1..*</code>	One or more
<code>0..1</code>	Zero or one (optional)
<code>2..5</code>	Between 2 and 5

Example: `Student "1..*" -- "1" Course` means one or more students per course, and each student enrollment links to exactly one course.

4. Examiner Viva Questions:

- Difference between Aggregation and Composition?
- What is multiplicity? What does 1..* mean?
- What notation is used for Generalization?

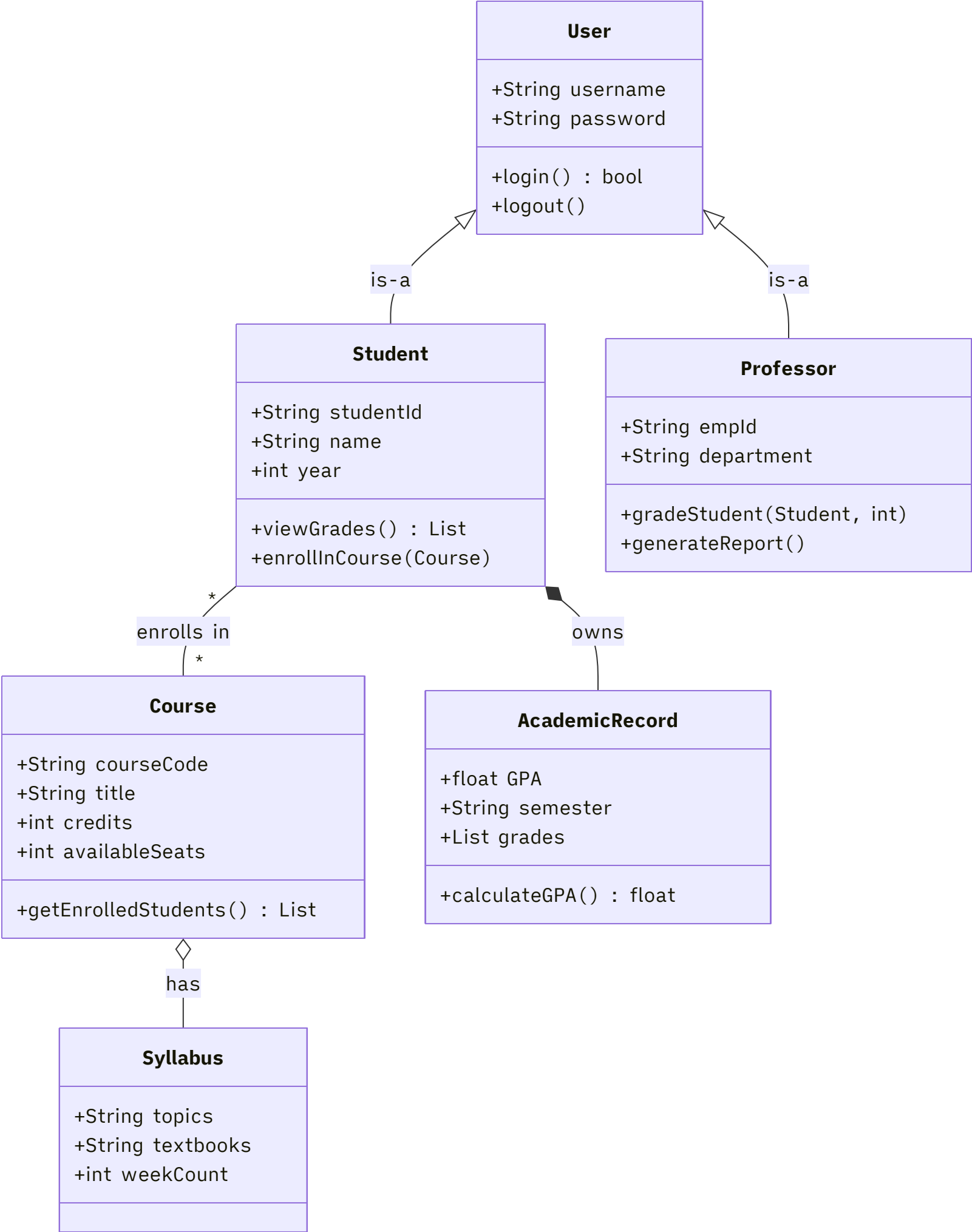
5. Strong Viva Answers:

- Answer: In Aggregation, the child (part) can exist independently of the parent (whole). In Composition, the child is completely dependent — it is created and destroyed with the parent.
- Answer: Multiplicity defines how many instances of one class relate to another. 1..* means "one or more."

9. Difference Tables:

Feature	Aggregation	Composition
Strength	Weak (HAS-A)	Strong (OWNS-A)
Lifetime of part	Independent	Dependent on whole
Diamond	Hollow ◇	Filled ◆
Real example	Course & Syllabus	Order & OrderItems
Human body	Body & Phone (external)	Body & Heart (internal)

📊 Visual: Class Diagram — All 4 Relationships



💡 **Exam tip:** Hollow diamond = Aggregation (weak). Filled diamond = Composition (strong). Hollow triangle = Generalization. Examiners will test you on this!

12. Oral Trap Questions:

- *Examiner:* "If I delete a **Course**, does **Syllabus** get deleted?"
- *Answer:* "No, sir. Course and Syllabus have an Aggregation relationship — the Syllabus exists independently. It would be deleted only in a Composition."
- *Examiner:* "What's the multiplicity between Student and AcademicRecord?"
- *Answer:* "1 to 1, sir. Each student has exactly one academic record."

13. Practical File Expectations:

- Show all 4 relationship types in your class diagram.
- Always show multiplicity on association lines.
- Use the 3-compartment class box: Name | Attributes | Methods.

Topic 5: Forward Engineering (Use Case Realization)

1. Simple Exam Definition:

Forward Engineering is the process of converting Use Case diagrams into Design elements (Sequence Diagrams, Collaboration Diagrams, and ultimately Class Diagrams) — i.e., tracing from requirement to design.

2. Core Concept Explanation:

This is how you go from "what the user wants" to "how the system provides it":

1. Use Case Diagram → Identify actors and use cases.
2. Use Case Specification → Write formal flows.
3. **Use Case Realization** → Create Sequence Diagram showing how objects collaborate to realize the use case.
4. Sequence Diagram → Extract participating classes → Add to Class Diagram.

3. Real Practical Example — Realizing "EnrollInCourse":

- **Use Case:** Student enrolls in a course.
- **Realization (Sequence Diagram):** Student → EnrollmentController → CourseService → CourseDAO → Database.
- **Classes extracted:** **EnrollmentController**, **CourseService**, **CourseDAO** → Added to Class Diagram.

4. Examiner Viva Questions:

- What is Use Case Realization?
- What is Forward Engineering?

5. Strong Viva Answers:

- *Answer:* Use Case Realization is the process of showing how a use case is implemented through collaborating objects, typically modeled as a Sequence or Collaboration Diagram.
- *Answer:* Forward Engineering moves from high-level Use Case requirements to detailed Design Diagrams and code.

